

AI Job Search & Application Assistant

Project Documentation — Built with LangGraph

1. Purpose of This Project

This is a personal AI assistant that helps you search for jobs, checks how well each job matches your resume, keeps track of the jobs you have applied to, and remembers your preferences over time. It does not apply to jobs automatically — it only helps you search, understand, and track.

This project is also how you learn LangGraph properly — by building something real instead of only watching tutorials.

2. Rule: Everything Must Be Free

Every tool, model, and API used in this project must be free (or a free tier). No paid subscriptions or paid API keys are required at any stage.

3. Tools You Will Use (all free)

- LLM (the AI brain): a free-tier model API such as Google Gemini (free tier) or Groq (free, very fast) — or a local free model through Ollama if you prefer running it on your own computer.
- Search tool (to find real job postings): Tavily Search API free tier, or DuckDuckGo search (no key needed, fully free).
- Framework: LangGraph (free, open-source Python library) and LangChain where useful.
- Storage for tracked jobs and memory: SQLite — a free, built-in database file, no server or account needed.
- Observability (seeing what the AI did step by step): LangSmith free tier, or skip this at first and add it later once the basic project works.
- Code editor / AI coding help: your existing Claude Code or similar tool, used to help you write and review code.

4. How the Project Is Organized (Milestones)

Go through these one at a time, in order. Do not skip ahead. Each milestone builds on the one before it. There is no fixed deadline — take as long as each step needs.

Milestone (a): Read the Resume

What to do:

- Let the assistant accept your resume (as text or a simple file).
- Have it pull out your skills, projects, and education in a clean, organized way.

How to do it:

- Write one LangGraph node whose only job is: take resume text in, give organized skill/experience info out.
- Test it by hand first — paste your resume, check the output makes sense.

Free tools to use: Free LLM API (Gemini/Groq) to read and organize the resume text.

Milestone (b): Basic Job Search

What to do:

- Take a simple request like "AI internship, remote" and return a small list of real job postings.

How to do it:

- Add a node that calls the free search tool with your request.
- Just get real results showing on screen first — do not worry about scoring yet.

Free tools to use: Tavily free tier or DuckDuckGo search — no payment needed.

Milestone (c): Score Each Job Match

What to do:

- For every job found, compare it to your resume.
- Give a score and a short reason, e.g. "Strong match — you have the RAG experience they ask for."

How to do it:

- Add a node that takes one job + your resume info, and asks the AI model to score and explain the match.
- Run this for each job found in milestone (b).

Milestone (d): Try Again if Matches Are Weak

What to do:

- If most jobs score low, automatically search again with different or broader words instead of stopping.

How to do it:

- This is your first conditional loop: after scoring, check the scores. If they are mostly low, send the flow back to the search step with adjusted search words. If good, move forward.

Milestone (e): Track Applications

What to do:

- Let yourself mark a job as "applied".
- Let yourself update its status later: interview, rejected, no response.

How to do it:

- Add a simple way to record and update status for each job (a small table of job + status).

Milestone (f): Save Everything Between Sessions (Persistence)

What to do:

- Make sure that if you close the app and come back later, nothing is lost — your tracked jobs, your resume info, everything.

How to do it:

- Connect LangGraph's memory-saving feature to a SQLite file. This file stores your progress on your computer, for free, with no account.

Free tools to use: SQLite (free, built into Python).

Milestone (g): Remember Your Preferences Over Time

What to do:

- Notice patterns, like which kinds of jobs you keep applying to vs skipping, and use that to improve future suggestions.

How to do it:

- Store a simple ongoing summary of your preferences, and update it occasionally based on your applied/skipped jobs.
- Feed this summary back into the scoring step from milestone (c).

Milestone (h): Show Progress as It Works (Streaming)

What to do:

- Instead of a long silent wait, show small updates: "Searching...", "Found 6 jobs, scoring now..."

How to do it:

- Turn on LangGraph's streaming output and print/display each step as it happens.

Milestone (i): See What Happened at Each Step (Observability)

What to do:

- Be able to look back and see exactly what the AI did, what it searched for, and why it scored a job the way it did.

How to do it:

- Connect LangSmith's free tier, or simply keep clear logs/print statements at first if you want to stay fully free without any sign-up.

Free tools to use: LangSmith free tier (optional sign-up) or plain logging.

5. What To Do When You Get Stuck

- Re-read the milestone you are on in this document — check exactly what it asks for, nothing more.
- Do not jump ahead to a later milestone to "fix" something — most confusion comes from mixing steps together too early.
- Ask your AI coding assistant to explain the specific error in simple words, and to walk you through it step by step, not just hand you a fixed version.
- If a whole milestone feels confusing, go back one milestone and make sure you can explain that one in your own words first.

6. Simple Glossary (Plain-Language Reminders)

- Node — one single step or task in the process (e.g. "search for jobs").
- Edge — the connection deciding what happens next after a node finishes.
- Conditional edge — a decision point: "if this, go here; if that, go there."
- State — the information being carried along and updated as the process runs.
- Subgraph — a smaller, self-contained group of nodes treated as one unit inside the bigger project.
- Persistence — saving progress so it is not lost when you close the app.
- Short-term memory — remembering things only during the current run.
- Long-term memory — remembering things across many days or sessions.
- Streaming — showing progress as it happens, instead of only the final result.
- Observability — being able to see and understand what the AI did at each step.